

Using Codex with the Customer 360 Database

Simple Project Handout: Zain Customer Churn Risk Mini Dashboard

Purpose: show students how a coding agent can inspect a small project folder, understand a SQLite database, create a focused Streamlit app, improve it step by step, and avoid large token usage.

1. What We Are Building

The project is a small Streamlit dashboard that reads the Zain Customer 360 SQLite database and helps a customer-care or retention manager identify high-risk churn customers.

- Use the full synthetic Customer 360 database, but keep the task small.
- Ask Codex to inspect the schema before writing code.
- Build one useful feature first: high-risk churn customer dashboard.
- Add customer details and rule-based recommendation only after the first version works.
- Add optional OpenAI-generated summary only at the end, and only for one selected customer.

Important: Use synthetic or training data only. Do not give real customer PII or confidential production data to coding agents during class.

2. Do Students Need ChatGPT Plus?

For VS Code or local Codex usage, students can sign in with either a ChatGPT account or an OpenAI API key. API key usage is billed through the OpenAI Platform account. Some Codex Cloud or ChatGPT-credit features may require ChatGPT sign-in instead of API-key mode. For this classroom demo, API-key mode is enough if the task is small and controlled.

3. Recommended Project Folder

```
zain-codex-demo/  
|  
■■■ app.py  
■■■ db_utils.py  
■■■ requirements.txt  
■■■ README.md  
|  
■■■ data/  
■■■ zain_customer_360_ai_demo.db
```

4. Recommended Classroom Flow

Time	Activity	Purpose
3 min	Show project folder	Students see the database and empty app files.
5 min	Ask Codex to inspect schema	Prevent blind code generation.
8 min	Ask Codex to build first Streamlit app	Create a small working dashboard.
5 min	Run and debug	Show Codex as a debugging partner.
7 min	Add customer details panel	Show iterative improvement.
5 min	Add rule-based recommendation	Show business logic before AI.
5 min	Optional AI summary	Use model only for one selected customer, not the whole DB.

5. Prompt 1 - Inspect the Project First

Use this prompt before asking Codex to write any code. This keeps token usage lower and prevents Codex from building the wrong thing.

```
Inspect this project folder.

There is a SQLite database at:
data/zain_customer_360_ai_demo.db

Do not write code yet.

First:
1. List the tables in the database.
2. Identify which tables are useful for a churn-risk mini dashboard.
3. Suggest a very small app structure using Streamlit.
4. Keep the scope simple.
5. Tell me which joins may be needed.

Important:
- Do not build a full enterprise app.
- Do not add RAG, agents, MCP, or OpenAI calls yet.
- Focus only on a small churn-risk dashboard.
```

Expected Codex Output

- A table list or schema summary.
- Suggested useful tables such as customers, customer_churn_scores, customer_value_segments, complaints, invoices, subscriptions, and plans.
- A small proposed app structure.
- A warning if table or column names need to be verified.

6. Prompt 2 - Create the First Streamlit Version

```
Create a simple Streamlit app in app.py.

Use the SQLite database at:
data/zain_customer_360_ai_demo.db

The app should:
1. Connect to the SQLite database.
2. Show a title: Zain Customer Churn Risk Dashboard.
3. Query high-risk churn customers.
4. Display customer_id, full_name, city, customer_segment, churn_score, risk_level,
main_risk_reason, and recommended_action.
5. Add a city filter if city exists.
6. Add a risk_level filter if risk_level exists.
7. Keep the code beginner-friendly.
8. Add comments.
9. Do not add AI or OpenAI calls yet.

If a table or column name is different, inspect the schema and adjust the query.
```

Teaching Point

The first version should be boring but working. This is the most important habit when using Codex: build the smallest useful feature first.

7. Prompt 3 - Add requirements.txt

```
Create requirements.txt for this project.

The app currently uses:
- streamlit
- pandas
- sqlite3 from Python standard library

Add only the packages that are required.
Keep the requirements file minimal.
```

```
streamlit
pandas
```

8. Prompt 4 - Add Customer 360 Detail Panel

```
Improve the app.

When the user selects a customer_id, show a Customer 360 summary.

Use available tables from the SQLite database.

Include:
1. Customer profile
2. Churn risk
3. Value segment
4. Recent complaints
5. Recent invoices or billing summary if available
6. Current subscription and plan if available

Keep it simple.

Use separate helper functions:
- get_customer_profile(customer_id)
- get_customer_churn_risk(customer_id)
- get_customer_value_segment(customer_id)
- get_customer_complaints(customer_id)
- get_customer_billing_summary(customer_id)
- get_customer_plan(customer_id)

If some tables or columns are missing, handle it gracefully and show a clear message.
```

Suggested Code Organization

At this stage, ask Codex to move database logic into helper functions. This keeps the app easier to explain to beginners.

```
app.py
- Streamlit UI
- filters
- selected customer view

db_utils.py
- database connection
- SQL helper functions
- customer 360 queries
```

9. Prompt 5 - Add Rule-Based Recommendation

```
Add a simple rule-based retention recommendation.

Rules:
- If risk_level is High and value segment is high-value, VIP, or premium, recommend priority retention call.
- If billing complaints exist, recommend billing clarification follow-up.
- If recent complaints exist, recommend service recovery action.
- If customer has high value but medium churn risk, recommend proactive care check-in.
- Otherwise, recommend normal monitoring.

Do not call any LLM.
Keep the logic deterministic and beginner-friendly.

Show the recommendation in a clear section called:
Recommended Retention Action
```

Why This Step Matters

This teaches that not every feature needs an LLM. Some business logic should be deterministic, cheaper, and easier to audit.

10. Prompt 6 - Ask Codex to Debug

```
Run the app and fix any errors.

Please:
1. Read the terminal error carefully.
2. Identify the root cause.
3. Make the smallest possible fix.
```

4. Do not rewrite the whole app.
5. Explain what was wrong and what you changed.

Keep the app simple and beginner-friendly.

11. Prompt 7 - Improve the UI Without Overbuilding

```
Improve the Streamlit layout.

Add:
1. Page title and short description
2. Sidebar filters for city and risk_level
3. Metric cards:
- total customers shown
- number of high-risk customers
- average churn score if available
4. Customer detail section
5. Recommended retention action section

Do not add charts unless the code is already working.
Do not add authentication.
Do not add OpenAI calls yet.
```

12. Prompt 8 - Optional AI Retention Summary

Use this only if you want to show an AI extension. This prompt controls token usage by sending only one selected customer summary, not the whole database.

```
Add an optional OpenAI-generated retention summary.

Use OPENAI_API_KEY from environment variable.
Use model: gpt-4.1-mini.

Important:
- Do not send the full database to the model.
- Only send the selected customer's summarized profile, churn risk, value segment,
  complaints, billing summary, and plan information.
- Add a button called: Generate AI Retention Summary.
- The AI summary should include:
1. Customer situation
2. Churn risk explanation
3. Recommended retention action
4. Short customer-care message

Keep the prompt short and safe.
If OPENAI_API_KEY is missing, show a clear message.
```

13. Prompt 9 - Create README.md

```
Create a beginner-friendly README.md.

Include:
1. Project title
2. What the app does
3. Folder structure
4. How to install requirements
5. How to run the app
6. What database file is expected
7. Example questions/use cases
8. What can be improved next

Keep it short and clear for students.
```

14. Prompt 10 - Prepare Final Demo Explanation

```
Write a short explanation for presenting this project to beginners.

Explain:
1. What problem the app solves
2. Which database is used
3. Which tables are important
4. How Streamlit displays the result
5. How the dashboard helps a retention manager
6. How this can later become an AI agent, RAG app, or MCP tool

Keep it under 2 minutes.
```

15. Token-Saving Rules for Students

- Do not ask Codex to build the full app in one prompt.
- Ask Codex to inspect first, then build one feature.
- Keep prompts specific and limited.
- Run the app after every major change.
- Ask for the smallest possible fix when errors happen.
- Avoid adding AI, RAG, MCP, dashboards, charts, and deployment in the same prompt.
- Only send one selected customer summary to the LLM if you add AI later.
- Use synthetic training data only.

16. What Not To Ask Codex

Avoid these prompts because they are too broad and will likely use more tokens or create messy code:

Bad prompt:
Build a complete AI app using the full Customer 360 database.

Bad prompt:
Analyze the entire database and build dashboards for everything.

Bad prompt:
Create a full enterprise AI copilot with RAG, SQL agents, MCP, LangSmith, and deployment.

Bad prompt:
Read the whole database and send everything to OpenAI.

Better Prompt Pattern

Good prompt:
Build one small feature first.

Use this database:
data/zain_customer_360_ai_demo.db

Goal:
Show high-risk churn customers in a simple Streamlit dashboard.

Do not add AI yet.
Keep the code beginner-friendly.

17. Final Student Checklist

Item	Done
Project folder created	■
Database placed in data/zain_customer_360_ai_demo.db	■
Codex inspected schema before writing code	■
First Streamlit dashboard works	■
Customer 360 detail panel added	■
Rule-based recommendation added	■
App tested with at least 3 customers	■
README.md created	■
Optional AI summary added only after app works	■

18. Suggested Extension Ideas

- Add a simple bar chart for high-risk customers by city.
- Add export to CSV for filtered customers.
- Add a customer priority score based on churn risk and value segment.
- Add AI-generated retention summary for one selected customer.
- Convert helper functions into LangChain tools.
- Expose selected helper functions through MCP.
- Create a Hugging Face Space version after the app is stable.

19. References

This handout is based on the current OpenAI Codex documentation: [Codex IDE extension](#), [Codex quickstart](#), [Codex authentication](#), and [Codex CLI/App guidance](#). Always check the latest OpenAI documentation before a live workshop because authentication, billing, and feature availability can change.